

Starthilfe: Entwicklungsumgebung einrichten

Python, VS Code und Streamlit – Schritt fuer Schritt

Robotron Bildungszentrum Halle

Fuer wen dieses Dokument ist

Dieses Dokument fuehrt Sie Schritt fuer Schritt durch die Einrichtung Ihrer Entwicklungsumgebung fuer das erste Python-Projekt. Sie koennen es am Vorabend oder ganz zu Beginn von Tag 1 durcharbeiten.

Am Ende haben Sie:

- Python installiert und im Terminal erreichbar
- Visual Studio Code eingerichtet mit Python-Unterstuetzung
- Einen Projektordner mit virtueller Umgebung
- Alle benoetigten Bibliotheken installiert
- Ein laufendes Hallo-Welt-Streamlit-Programm
- Ein Testskript, das eine erste API-Anfrage stellt

Zeitbedarf

Installationen (einmalig): ca. 30–45 Minuten

Hallo-Welt und API-Test: ca. 15–20 Minuten

Gesamt: ca. 1 Stunde

Das Fundament steht danach ein fuer alle Mal.

Contents

1	Das grosse Bild: Was wir einrichten	3
2	Schritt 1: Python pruefen und installieren	3
2.1	Pruefen: Ist Python schon da?	3
2.2	Python herunterladen und installieren	4
3	Schritt 2: Visual Studio Code einrichten	5
3.1	VS Code herunterladen und installieren	5
3.2	Die Python-Erweiterung installieren	5
4	Schritt 3: Projektordner anlegen und in VS Code oeffnen	6
4.1	Das integrierte Terminal oeffnen	6
5	Schritt 4: Virtuelle Umgebung anlegen und aktivieren	7
5.1	Warum eine virtuelle Umgebung?	7
5.2	Virtuelle Umgebung erstellen	7
5.3	Virtuelle Umgebung aktivieren	7
5.4	Python-Interpreter in VS Code auswaehlen	8
6	Schritt 5: Bibliotheken installieren	9
7	Schritt 6: Schutzfiles anlegen	10
7.1	.gitignore: Was nicht ins Versionsverwaltungssystem gehoert	10
7.2	.env und .env.example: Konfiguration ausserhalb des Codes	11
8	Schritt 7: Projektstruktur anlegen	11
9	Schritt 8: Hallo Welt mit Streamlit	12
9.1	Die app.py bearbeiten	12
9.2	Streamlit starten	13

10 Schritt 9: Erste API-Anfrage testen	14
10.1 Testskript fuer Variante A: Strompreise (aWATTar)	14
10.2 Testskript fuer Variante B: Wetter (Open-Meteo)	15
10.3 Testskript ausfuehren	16
 11 Startcheckliste	 17

1 Das grosse Bild: Was wir einrichten

Bevor wir loslegen, ein kurzer Ueberblick ueber alle Werkzeuge und warum wir jedes davon brauchen.

Werkzeug	Was es ist	Warum wir es brauchen
Python 3.14	Die Programmiersprache	Ohne Python laeuft kein Python-Code
Visual Studio Code	Editor mit Erweiterungen	Komfortables Schreiben, Autocomplete, integriertes Terminal
Python-Erweiterung	Erweiterung fuer VS Code	Syntaxfarben, Fehlerhinweise, Debugger
venv	Virtuelle Umgebung	Jedes Projekt hat eigene Bibliotheken – kein Versionswirrwarr
pip	Paketverwaltung	Bibliotheken wie streamlit oder pandas installieren

Hinweis

Betriebssystem: Diese Anleitung ist primaer fuer **Windows** geschrieben. An den Stellen, wo Linux oder macOS abweicht, ist das ausdrucklich angegeben.

2 Schritt 1: Python pruefen und installieren

2.1 Pruefen: Ist Python schon da?

Oeffnen Sie die Windows-Eingabeaufforderung:

1. Tastenkombination **Win+R** druecken.
2. `cmd` eingeben und Enter druecken.
3. In das schwarze Fenster tippen:

Python-Version pruefen

```
1 python --version
```

Moegliche Ergebnisse:

- **Python 3.10.x** oder hoeher – gut, Python ist vorhanden. Weiter mit Abschnitt 3.

- Python 3.9.x oder älter – veraltet, bitte neu installieren (Abschnitt 2.2).
- Fehlermeldung oder `'python' is not recognized` – Python fehlt oder ist nicht im PATH. Weiter mit Abschnitt 2.2.

Hinweis

Manchmal ist Python unter dem Befehl `python3` erreichbar. Probieren Sie in dem Fall `python3 -version`.

2.2 Python herunterladen und installieren

1. Öffnen Sie Ihren Browser und gehen Sie zu: <https://www.python.org/downloads/>
2. Klicken Sie auf den grossen gelben Download-Button fuer Python 3.14 (oder die aktuell empfohlene stabile Version).
3. Starten Sie die heruntergeladene `.exe`-Datei.

Wichtig!

Diesen Haken nicht vergessen!

Ganz unten im Installationsfenster – noch bevor Sie auf **“Install Now”** klicken – gibt es eine Option:

☒ Add Python to PATH
oder
☒ Add python.exe to PATH

Setzen Sie diesen Haken. Ohne ihn findet Windows Python nicht, und alle nachfolgenden Befehle scheitern.

4. Klicken Sie auf **“Install Now”**.
5. Warten Sie, bis die Installation abgeschlossen ist.
6. **Schliessen Sie die Eingabeaufforderung und öffnen Sie eine neue.** Die alte kennt die neue Python-Installation noch nicht.
7. Kontrollieren Sie erneut: `python -version`

Checkpoint: Die Ausgabe von `python -version` zeigt Python 3.10.x oder eine höhere Versionsnummer? Dann ist Python korrekt installiert.

3 Schritt 2: Visual Studio Code einrichten

Visual Studio Code (kurz: VS Code) ist ein kostenloser Editor von Microsoft. Er ist in der Praxis weit verbreitet und unterstützt Python hervorragend.

3.1 VS Code herunterladen und installieren

1. Gehen Sie zu: <https://code.visualstudio.com/>
2. Klicken Sie auf **“Download for Windows”**.
3. Starten Sie den Installer.
4. Empfehlung bei der Installation: Setzen Sie die Haken bei
 - *“Code zum PATH hinzufuegen”* (nach Neustart verfuegbar)
 - *“Mit Code oeffnen” – Aktion im Kontextmenue* (praktischer Rechtsklick auf Ordner)
5. Schliessen Sie die Installation ab und starten Sie VS Code.

3.2 Die Python-Erweiterung installieren

VS Code ist ein allgemeiner Editor. Fuer Python brauchen wir eine Erweiterung:

1. Klicken Sie in der linken Leiste auf das **Erweiterungs-Symbol** (vier kleine Quadrate, eines davon herausgeloesst) – oder Tastenkombination **Strg+Shift+X**.
2. In das Suchfeld tippen: **Python**
3. Die erste Erweiterung in der Liste ist **“Python”** von **Microsoft**. Klicken Sie auf **“Install”**.
4. Warten Sie, bis die Installation abgeschlossen ist.

Praxistipp

Optional, aber empfehlenswert: Installieren Sie auch **“Pylance”** (ebenfalls von Microsoft). Es verbessert die Autocomplete-Vorschlaege erheblich. Suchen Sie es genauso wie die Python-Erweiterung.

Checkpoint: VS Code ist geoeffnet und zeigt unten in der blauen Statusleiste eine Python-Versionsnummer? Dann ist die Erweiterung aktiv. Falls dort noch nichts steht: erst nach dem naechsten Schritt (Interpreter auswahlen) sehen Sie sie.

4 Schritt 3: Projektordner anlegen und in VS Code öffnen

Jedes Projekt bekommt einen *eigenen Ordner*. Alle Dateien – Code, Datenbank, Konfiguration – landen darin.

1. Legen Sie einen neuen Ordner auf Ihrem Laufwerk an, zum Beispiel:

Beispielpfade (wählen Sie einen)

```
C:\Projekte\strompreise  
C:\Projekte\staedte-dashboard  
C:\Users\IhrName\Dokumente\Projekt1
```

2. Öffnen Sie VS Code.
3. Klicken Sie auf **Datei > Ordner öffnen...** (oder **Strg+K**, dann **Strg+O**).
4. Navigieren Sie zu Ihrem neuen Ordner und klicken Sie auf **“Ordner auswählen”**.

VS Code öffnet jetzt diesen Ordner. Im linken Explorer-Bereich sehen Sie den Ordernamen – er ist noch leer. Das ist korrekt.

Wichtig!

Verzeichnisnamen ohne Leerzeichen und Umlaute verwenden. Namen wie `mein projekt` oder `Staedte Dashboard` koennen spaeter im Terminal zu schwer zu debuggenden Problemen fuehren.

Gut: `mein-projekt` Gut: `staedte_dashboard`

Schlecht: `mein projekt` Schlecht: `Städte Dashboard`

4.1 Das integrierte Terminal öffnen

Ab jetzt arbeiten wir mit dem Terminal *direkt in VS Code*. Das hat einen grossen Vorteil: Das Terminal startet automatisch im Projektordner.

1. Klicken Sie auf **Terminal > Neues Terminal** in der Menueleiste.
2. Unten in VS Code öffnet sich ein Terminalbereich (PowerShell oder Eingabeaufforderung).
3. Der Pfad in der Eingabezeile zeigt bereits Ihren Projektordner.

Praxistipp

Tastenkuerzung: **Strg** + **`** (das Gravis-Zeichen, auf deutschen Tastaturen mit Shift+Accent oder ueber die Taste links neben der 1) oeffnet und schliesst das Terminal. Diese Kombination werden Sie die naechsten Tage sehr haeufig benutzen.

5 Schritt 4: Virtuelle Umgebung anlegen und aktivieren

5.1 Warum eine virtuelle Umgebung?

Definition

Eine **virtuelle Umgebung** (kurz: *venv*) ist ein abgeschlossener Python-Bereich fuer ein einzelnes Projekt. Bibliotheken, die Sie darin installieren, sind nur fuer dieses Projekt sichtbar und beeinflussen kein anderes Projekt auf dem gleichen Rechner.

Ohne *venv* werden alle Pakete global installiert. Das fuehrt fruehzeitig zu Versionskonflikten: Projekt A braucht pandas 1.5, Projekt B braucht pandas 2.0 – was gewinnt? Mit einer virtuellen Umgebung stellt sich die Frage gar nicht, weil jedes Projekt seine eigene, saubere Welt hat.

5.2 Virtuelle Umgebung erstellen

Im Terminal in VS Code (der Projektordner ist bereits aktiv):

Virtuelle Umgebung anlegen

```
1 python -m venv .venv
```

Was passiert:

- Python erstellt einen Ordner *.venv* in Ihrem Projektordner.
- Darin befindet sich eine vollstaendige, separate Python-Installation.
- Der Befehl gibt kein sichtbares Feedback – das ist normal. Er dauert einige Sekunden.

Hinweis

Der Punkt vor *venv* (also *.venv*) macht den Ordner auf Linux und macOS versteckt. Auf Windows ist er sichtbar. Beides ist korrekt.

5.3 Virtuelle Umgebung aktivieren

Das Erstellen allein genuegt nicht – die Umgebung muss auch *aktiviert* werden.

Windows – PowerShell (Standard in VS Code):

venv aktivieren (Windows PowerShell)

```
1 .venv\Scripts\activate
```

Windows – Eingabeaufforderung (cmd):

venv aktivieren (Windows cmd)

```
1 .venv\Scripts\activate.bat
```

Linux und macOS:

venv aktivieren (Linux / macOS)

```
1 source .venv/bin/activate
```

Erkennungszeichen der aktivierten Umgebung: Nach der Aktivierung steht `(.venv)` am Anfang jeder Terminalzeile:

So sieht ein aktiviertes venv aus

```
(.venv) PS C:\Projekte\strompreise>
```

Wichtig!

PowerShell-Sicherheitsrichtlinie: Auf manchen Windows-Computern erscheint beim ersten Aktivieren die Fehlermeldung *“cannot be loaded because running scripts is disabled on this system”*.

Loesung: Folgenden Befehl einmalig in PowerShell eingeben:

```
1 Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope  
   CurrentUser
```

Danach `.venv\Scripts\activate` erneut ausfuehren.

5.4 Python-Interpreter in VS Code auswaehlen

VS Code muss wissen, welches Python es verwenden soll – das aus dem venv.

1. Tastenkombination **Strg+Shift+P** druecken (oeffnet die Befehlspalette).

2. Tippen Sie: Python: Select Interpreter
3. Enter druecken.
4. In der Liste erscheinen mehrere Python-Installationen. Waehlen Sie diejenige, die `.venv` im Pfad enthaelt:

```
Python 3.14.x ('.venv': venv)
.\.venv\Scripts\python.exe
```

5. Klicken Sie darauf.

Ab jetzt aktiviert VS Code neue Terminals automatisch mit dem `venv`.

Checkpoint: Im Terminal steht `(.venv)` vor dem Prompt? Unten rechts in VS Code erscheint die Python-Versionsnummer mit `'venv'` in Klammern? Dann ist alles korrekt eingestellt.

6 Schritt 5: Bibliotheken installieren

Jetzt installieren wir alle Bibliotheken, die das Projekt benoetigt. **Das `venv` muss aktiv sein** – Sie erkennen es am `(.venv)` im Terminal.

Alle Projektbibliotheken auf einmal installieren

```
1 pip install streamlit pandas requests python-dotenv openpyxl
```

Was die einzelnen Pakete machen:

- `streamlit` – Das grafische Frontend: aus Python-Skripten werden automatisch Webseiten.
- `pandas` – Tabellendaten verarbeiten, filtern, auswerten.
- `requests` – HTTP-Anfragen an externe APIs stellen.
- `python-dotenv` – Konfigurationswerte aus einer `.env`-Datei einlesen.
- `openpyxl` – Excel-Dateien lesen und schreiben (Alternative zu SQLite, falls gewaehlt).

Die Installation dauert 1–3 Minuten. Sie sehen viele Zeilen Ausgabe – das ist normal und kein Fehler.

Anschliessend speichern Sie den exakten Zustand aller Pakete:

requirements.txt erzeugen

```
1 pip freeze > requirements.txt
```

Diese Datei listet alle installierten Pakete mit Versionsnummern. Wer Ihr Projekt später frisch einrichten will – oder Sie selbst auf einem anderen Rechner – kann mit `pip install -r requirements.txt` exakt dieselbe Umgebung wiederherstellen.

Checkpoint: Im Projektordner gibt es eine Datei `requirements.txt`? Wenn Sie sie öffnen, sehen Sie Zeilen wie `streamlit==1.x.x` und `pandas==3.x.x`? Dann war die Installation erfolgreich.

7 Schritt 6: Schutzfiles anlegen

Zwei Dateien legen wir von Anfang an an – nicht am Ende, wenn es schiefgelaufen ist.

7.1 .gitignore: Was nicht ins Versionsverwaltungssystem gehoert

Eine `.gitignore`-Datei teilt Git mit, welche Dateien es ignorieren soll. Das betrifft automatisch generierte Dateien, sensible Zugangsdaten und gerätespezifische Konfigurationen.

Legen Sie im Projektordner eine neue Datei **.gitignore** an:

1. Im Explorer links in VS Code: Klicken Sie auf das Symbol *Neue Datei* (oder Rechtsklick > Neue Datei).
2. Dateiname: `.gitignore` (genau so – mit Punkt am Anfang, ohne Dateiendung).
3. Schreiben Sie folgenden Inhalt:

Inhalt der .gitignore

```
1 # Virtuelle Umgebung -- nie ins Git!
2 .venv/
3
4 # Konfigurationsdatei (kann Zugangsdaten enthalten)
5 .env
6
7 # Python-Bytecode (automatisch generiert)
8 __pycache__/*
9 *.pyc
```

```
10 *.pyo
11
12 # Datenbankdatei (enthaelt Nutzerdaten)
13 *.db
14
15 # Betriebssystem-Hilfsdateien
16 .DS_Store
17 Thumbs.db
```

7.2 .env und .env.example: Konfiguration ausserhalb des Codes

Einstellungen – und spaeter auch API-Schluessel – gehoeren nicht direkt in den Python-Code. Sie kommen in eine `.env`-Datei.

Datei 1: `.env` (wird nicht ins Git hochgeladen, steht in `.gitignore`)

Inhalt von `.env`

```
1 # Lokale Konfiguration -- nicht weitergeben!
2 DB_PFAD=daten.db
3 API_TIMEOUT=10
```

Datei 2: `.env.example` (Vorlage fuer andere, kommt ins Git)

Inhalt von `.env.example`

```
1 # Vorlage -- kopieren und in .env umbenennen
2 DB_PFAD=daten.db
3 API_TIMEOUT=10
4 # MY_API_KEY=schluessel-hier-eintragen
```

Hinweis

Die APIs fuer dieses Projekt (aWATTar, Open-Meteo) benoetigen keinen Schluessel – Sie koennen die `.env` also erstmal so lassen wie sie ist. Der Aufwand, sie jetzt anzulegen, lohnt sich trotzdem: Wenn Sie irgendwann einen Schluessel benoetigen, ist der richtige Platz schon da.

8 Schritt 7: Projektstruktur anlegen

Bevor wir die erste Streamlit-App schreiben, legen wir alle Moduldateien als *leere Platzhalter* an. Lieber frueh eine saubere Struktur als spaeter alles in einer Datei.

Legen Sie im Projektordner folgende Dateien an (Rechtsklick > Neue Datei im VS-Code-

Explorer):

Projektstruktur (Variante A – Strompreise)

```
mein_projekt/  
|-- main.py          <- Einstiegspunkt  
|-- app.py           <- Streamlit-Frontend  
|-- api_client.py    <- API-Aufruf  
|-- datenbank.py     <- SQLite-Zugriff  
|-- logik.py         <- Berechnungen, pandas  
|-- .env             <- Konfiguration (schon vorhanden)  
|-- .env.example     <- Vorlage (schon vorhanden)  
|-- .gitignore       <- Ignorierregeln (schon vorhanden)  
|-- requirements.txt <- Paketliste (schon vorhanden)
```

Schreiben Sie in jede neue .py-Datei einen **Modul-Docstring** als Platzhalter:

Vorlage fuer jeden Modul-Docstring

```
1  """  
2  Modulname: api_client.py  
3  Beschreibung: API-Aufruf und Datenaufbereitung  
4  Autor: Vorname Nachname  
5  Datum: Juni 2026  
6  """
```

9 Schritt 8: Hallo Welt mit Streamlit

Jetzt schreiben wir das erste laufende Streamlit-Programm.

9.1 Die app.py bearbeiten

Oeffnen Sie `app.py` in VS Code und ersetzen Sie den Platzhalter-Docstring durch folgenden Inhalt:

Hallo-Welt-Streamlit (app.py – vollstaendiger Inhalt)

```
1  """  
2  app.py -- Streamlit-Frontend (Hallo-Welt-Version)  
3  Autor: Vorname Nachname  
4  Datum: Juni 2026  
5  """  
6  import streamlit as st  
7  
8  # Seitentitel und Layout festlegen
```

```
9 st.set_page_config(page_title="Mein Projekt", layout="wide")
10
11 # Hauptueberschrift
12 st.title("Hallo, Streamlit!")
13
14 # Einfache Eingabe
15 name = st.text_input("Wie heissen Sie?", placeholder="Ihr Name")
16
17 # Reaktion auf Eingabe
18 if name:
19     st.success(f"Willkommen, {name}! Die App laeuft.")
20
21 # Trennlinie und Hinweis
22 st.divider()
23 st.info("Diese App wird in den naechsten Tagen ausgebaut.")
```

Speichern Sie die Datei mit **Strg+S**.

9.2 Streamlit starten

Im Terminal (venv muss aktiv sein – (.venv) sichtbar):

Streamlit-App starten

```
1 streamlit run app.py
```

Was dann passiert:

1. Streamlit startet einen lokalen Webserver.
2. Im Terminal erscheint:

```
Local URL: http://localhost:8501
```

3. Ihr Browser oeffnet sich automatisch mit der App. Falls nicht: <http://localhost:8501> manuell im Browser eingeben.
4. Sie sehen eine Webseite mit einem Texteingabefeld.
5. Geben Sie Ihren Namen ein – es erscheint eine gruene Willkommensmeldung.

Praxistipp

Aenderungen sofort sehen: Wenn Sie die `app.py` speichern (**Strg+S**), erscheint in der Browser-App oben ein Banner *“Source file changed – Rerun”*. Klicken Sie darauf, um die Aenderungen zu sehen. Oder waehlen Sie *“Always rerun”* – dann aktualisiert die App bei jedem Speichern automatisch.

Wichtig!

Streamlit beenden: Druecken Sie im Terminal **Strg+C**. Das beendet den lokalen Webserver. Der Browser zeigt dann *“This site can’t be reached”* – das ist korrekt.

Checkpoint: Ihr Browser zeigt die Streamlit-App? Sie tippen einen Namen und sehen die gruene Willkommensmeldung? Ausgezeichnet – Streamlit laeuft!

10 Schritt 9: Erste API-Anfrage testen

Als letzten Schritt testen wir, ob wir die API unserer Projektvariante erreichen koennen. Das machen wir in einem separaten Testskript – noch nicht in der App.

Legen Sie eine neue Datei an – je nach Ihrer Projektvariante:

- Variante A (Strompreise): `test_api_strompreise.py`
- Variante B (Wetter): `test_api_wetter.py`

10.1 Testskript fuer Variante A: Strompreise (aWATTar)

test_api_strompreise.py

```
1  """
2  test_api_strompreise.py -- Schnelltest: aWATTar-API erreichbar
   ?
3  Dieses Skript ist nur fuer den ersten Verbindungstest (
   Variante A).
4  """
5  import requests
6
7  URL = "https://api.awattar.de/v1/marketdata"
8
```

```
9 print("Verbinde mit aWATTar API ...")
10
11 try:
12     antwort = requests.get(URL, timeout=10)
13     antwort.raise_for_status()
14     daten = antwort.json()
15
16     print(f"Verbindung erfolgreich! HTTP-Status: {antwort.status_code}")
17     print(f"Anzahl Preispunkte: {len(daten['data'])}")
18     print()
19     print("Erster Datenpunkt (rohe Ausgabe):")
20     print(daten["data"][0])
21     print()
22     print("Preis (Cent/kWh):", daten["data"][0]["marketprice"]
23           / 10)
24
25 except requests.exceptions.ConnectionError:
26     print("FEHLER: Keine Internetverbindung.")
27 except requests.exceptions.Timeout:
28     print("FEHLER: Zeitlimit ueberschritten (10 Sekunden).")
29 except requests.exceptions.HTTPError as e:
30     print(f"HTTP-FEHLER: {e}")
31 except Exception as e:
32     print(f"UNBEKANNTER FEHLER: {e}")
```

10.2 Testskript fuer Variante B: Wetter (Open-Meteo)

```
test_api_wetter.py
1 """
2 test_api_wetter.py -- Schnelltest: Open-Meteo-API erreichbar?
3 Dieses Skript ist nur fuer den ersten Verbindungstest (
4     Variante B).
5 """
6
7 import requests
8
9 URL = "https://api.open-meteo.com/v1/forecast"
10
11 # Berlin als Testkoordinaten
12 PARAMS = {
13     "latitude": 52.52,
14     "longitude": 13.41,
15     "current_weather": True,
16 }
```



```
16 print("Verbinde mit Open-Meteo API ...")
17
18 try:
19     antwort = requests.get(URL, params=PARAMS, timeout=10)
20     antwort.raise_for_status()
21     daten = antwort.json()
22
23     print(f"Verbindung erfolgreich! HTTP-Status: {antwort.status_code}")
24     print()
25     wetter = daten["current_weather"]
26     print("Aktuelles Wetter in Berlin:")
27     print(f"    Temperatur:           {wetter['temperature']} Grad C")
28     print(f"    Windgeschwindigkeit: {wetter['windspeed']} km/h")
29     print(f"    Wettercode:           {wetter['weathercode']}")
30     print(f"    Zeitpunkt:           {wetter['time']}")
31
32 except requests.exceptions.ConnectionError:
33     print("FEHLER: Keine Internetverbindung.")
34 except requests.exceptions.Timeout:
35     print("FEHLER: Zeitlimit ueberschritten (10 Sekunden).")
36 except requests.exceptions.HTTPError as e:
37     print(f"HTTP-FEHLER: {e}")
38 except Exception as e:
39     print(f"UNBEKANNTER FEHLER: {e}")
```

10.3 Testskript ausfuehren

Im Terminal (venv aktiv!):

Testskript starten (Variante A)

```
1 python test_api_strompreise.py
```

Testskript starten (Variante B)

```
1 python test_api_wetter.py
```

Bei Erfolg sehen Sie echte Daten aus dem Internet: Strompreise oder aktuelle Wetterwerte. Das belegt, dass Internetverbindung, Bibliothek und API-URL alle funktionieren.

Praxistipp

Nehmen Sie sich einen Moment, um die Ausgabe zu lesen. Was ist ein Dictionary? Was ist eine Liste? Wo steckt der eigentliche Preis – oder die Temperatur? Diese Struktur werden Sie morgen in `api_client.py` auspacken. Es lohnt sich, sie jetzt schon zu kennen.

Checkpoint: Das Testskript laeuft fehlerfrei und zeigt echte Daten?
Dann haben Sie alles, was Sie fuer den Projektstart brauchen.

11 Startcheckliste

Gehen Sie diese Liste durch, bevor Sie morgen mit dem echten Projektcode beginnen:

Alles bereit – Checkliste fuer den Start

- ☐ Python 3.10 oder hoeher ist installiert (`python -version` zeigt eine Versionsnummer)
- ☐ Visual Studio Code ist installiert
- ☐ Python-Erweiterung von Microsoft ist in VS Code installiert
- ☐ Projektordner ist angelegt und in VS Code geoeffnet
- ☐ Virtuelle Umgebung `.venv` ist angelegt
- ☐ Virtuelle Umgebung ist aktiv – (`.venv`) steht im Terminal
- ☐ Python-Interpreter in VS Code zeigt auf `.venv`
- ☐ Alle Bibliotheken installiert: `streamlit`, `pandas`, `requests`, `python-dotenv`, `openpyxl`
- ☐ `requirements.txt` ist vorhanden und aktuell
- ☐ `.gitignore` ist angelegt
- ☐ `.env` und `.env.example` sind angelegt
- ☐ Alle Modul-Dateien (`app.py`, `api_client.py` usw.) sind als Platzhalter angelegt
- ☐ Hallo-Welt-Streamlit laeuft im Browser
- ☐ Testskript (`test_api_strompreise.py` oder `test_api_wetter.py`) laeuft fehlerfrei und zeigt Daten

Hinweis

Wenn noch nicht alles klappt: Geben Sie bitte Bescheid. Diese Einrichtung ist einmalig – wenn sie steht, steht sie. Morgen fangen wir direkt mit dem echten Projektcode an.

Zusammenfassung

Heute haben Sie:

- Python und VS Code installiert und konfiguriert
- Gelernt, wozu eine virtuelle Umgebung gut ist und wie man sie anlegt
- Alle Projektbibliotheken installiert und in `requirements.txt` gesichert
- Die ersten Schutzfiles `.gitignore` und `.env` angelegt
- Eine vollstaendige Projektstruktur mit leeren Modulen angelegt
- Ein erstes laufendes Streamlit-Programm geschrieben und im Browser gesehen
- Eine echte API-Anfrage gestellt und die Antwortstruktur kennengelernt

Das Fundament steht. Morgen bauen wir darauf auf.